

Министерство образования Республики Беларусь
Учреждение образования «Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4

По дисциплине «Модели решения задач в ИС»

Тема: «Классификация линейно разделимых образов однослойным персептроном
для логических функций n переменных»

Выполнил:

Студент 3 курса

Группы ИИ-26 (2)

Яшков П.А.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: Реализовать нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнить эффективность обучения при различных начальных условиях и параметрах шага.

Задачи лабораторной работы:

1. Сгенерировать полную таблицу истинности для заданной логической функции (2^n наборов). На выходе функции должны использоваться значения 0 и 1.
2. **Реализовать алгоритм обучения:**
 - Разделить выборку на обучающую (часть данных) и тестовую (оставшиеся наборы) для проверки обобщающей способности.
 - В качестве функции активации использовать сигмоиду
 - Провести **два эксперимента** по обучению:
 - А. С фиксированным шагом обучения
 - Б. С адаптивным шагом
 - Выход сети интерпретировать как вероятность принадлежности к классу «1».
 - Обучение проводить до минимизации суммарной ошибки на обучающей выборке.
3. **Провести анализ сходимости:**
 - Построить графики зависимости суммарной ошибки от номера эпохи для обучающей и тестовой выборок.
 - Оценить обобщающую способность: корректно ли сеть предсказывает значения на тех наборах из таблицы истинности, которые она не видела при обучении.
 - Зафиксировать количество эпох, необходимых для настройки весов при разных значениях
4. Вывести итоговые значения весовых коэффициентов и порога.
5. **Реализовать режим функционирования:**
 - Программа должна принимать на вход вектор из n чисел (0 или 1).
 - Сеть должна мгновенно выдавать результат (вероятность и округленный класс).
6. Сделать вывод о способности сигмоидального персептрона представлять базовые логические операции и восстанавливать полную таблицу истинности по частичной выборке.

Вариант 25

№ of variant	n	Task
25	11	OR

Код программы:

```
import itertools
import numpy as np
import matplotlib.pyplot as plt
```

```

# =====
# НАСТРОЙКИ ЛАБЫ
# =====
N_INPUTS = 11
TRAIN_RATIO = 0.7
MAX_EPOCHS = 5000
RANDOM_SEED = 42

# Фиксированные шаги обучения для эксперимента
FIXED_LEARNING_RATES = [0.1, 0.3, 0.5]

# Начальный шаг для адаптивного обучения
ADAPTIVE_INITIAL_LR = 0.5

# =====
# ЛОГИЧЕСКАЯ ФУНКЦИЯ OR
# =====
def logical_or(x: np.ndarray) -> int:
    """
    Логическая функция OR для вектора из 0 и 1.
    Возвращает 1, если хотя бы один элемент равен 1, иначе 0.
    """
    return int(np.any(x == 1))

# =====
# ГЕНЕРАЦИЯ ТАБЛИЦЫ ИСТИННОСТИ
# =====
def generate_truth_table(n: int):
    """
    Генерирует полную таблицу истинности для n входов.
    X: все возможные комбинации 0/1 длины n
    y: значения функции OR
    """
    combinations = list(itertools.product([0, 1], repeat=n))
    X = np.array(combinations, dtype=float)
    y = np.array([logical_or(row) for row in X], dtype=float).reshape(-1, 1)
    return X, y

# =====
# РАЗДЕЛЕНИЕ НА TRAIN / TEST
# =====
def train_test_split_manual(X, y, train_ratio=0.7, seed=42):
    """
    Перемешивает данные и делит их на обучающую и тестовую выборки.
    """
    np.random.seed(seed)
    indices = np.arange(len(X))
    np.random.shuffle(indices)

```

```
split_index = int(len(X) * train_ratio)
train_idx = indices[:split_index]
test_idx = indices[split_index:]
```

```
X_train = X[train_idx]
y_train = y[train_idx]
X_test = X[test_idx]
y_test = y[test_idx]
```

```
return X_train, X_test, y_train, y_test
```

```
# =====
# СИГМОИДА И ОШИБКА
# =====
def sigmoid(z):
    """
    Сигмоидальная функция активации.
    """
    z = np.clip(z, -500, 500)
    return 1.0 / (1.0 + np.exp(-z))
```

```
def mse_loss(y_true, y_pred):
    """
    Среднеквадратичная ошибка.
    """
    return np.mean((y_true - y_pred) ** 2)
```

```
# =====
# ПЕРСЕЙТРОН
# =====
class SingleLayerPerceptron:
    def __init__(self, n_inputs: int):
        """
        Инициализация весов и порога (bias).
        """
        self.n_inputs = n_inputs
        self.weights = np.random.uniform(-0.5, 0.5, (n_inputs, 1))
        self.bias = np.random.uniform(-0.5, 0.5, (1,))

    def forward(self, X):
        """
        Прямой проход.
        """
        z = np.dot(X, self.weights) + self.bias
        return sigmoid(z)

    def predict_proba(self, X):
        """
        Вероятность принадлежности к классу 1.
        """
```

```

"""
return self.forward(X)

def predict(self, X):
    """
    Округленный класс 0 или 1.
    """
    probabilities = self.predict_proba(X)
    return (probabilities >= 0.5).astype(int)

def train_fixed_lr(self, X_train, y_train, X_test, y_test, learning_rate=0.1, max_epochs=1000):
    """
    Обучение с фиксированным шагом.
    Возвращает историю ошибок на train и test.
    """
    train_errors = []
    test_errors = []

    for epoch in range(max_epochs):
        # Прямой проход
        y_pred_train = self.forward(X_train)

        # Ошибка
        error = y_train - y_pred_train

        # Градиенты для MSE + sigmoid
        grad_output = error * y_pred_train * (1 - y_pred_train)

        grad_w = np.dot(X_train.T, grad_output) / len(X_train)
        grad_b = np.mean(grad_output, axis=0)

        # Обновление весов
        self.weights += learning_rate * grad_w
        self.bias += learning_rate * grad_b

        # Подсчет ошибок после обновления
        train_pred = self.forward(X_train)
        test_pred = self.forward(X_test)

        train_loss = mse_loss(y_train, train_pred)
        test_loss = mse_loss(y_test, test_pred)

        train_errors.append(train_loss)
        test_errors.append(test_loss)

        # Условие остановки
        if train_loss < 1e-5:
            break

    return train_errors, test_errors, epoch + 1

def train_adaptive_lr(self, X_train, y_train, X_test, y_test, initial_lr=0.5, max_epochs=1000):

```

"""

Обучение с адаптивным шагом.

Если ошибка уменьшается, шаг немного увеличивается.

Если ошибка растет, шаг уменьшается.

"""

train_errors = []

test_errors = []

learning_rate = initial_lr

prev_train_loss = float("inf")

for epoch in range(max_epochs):

Сохраняем старые параметры

old_weights = self.weights.copy()

old_bias = self.bias.copy()

Прямой проход

y_pred_train = self.forward(X_train)

error = y_train - y_pred_train

Градиенты

grad_output = error * y_pred_train * (1 - y_pred_train)

grad_w = np.dot(X_train.T, grad_output) / len(X_train)

grad_b = np.mean(grad_output, axis=0)

Пробное обновление

self.weights += learning_rate * grad_w

self.bias += learning_rate * grad_b

Новая ошибка

new_train_pred = self.forward(X_train)

train_loss = mse_loss(y_train, new_train_pred)

Адаптация шага

if train_loss < prev_train_loss:

learning_rate *= 1.05

else:

self.weights = old_weights

self.bias = old_bias

learning_rate *= 0.7

Повторное обновление с меньшим шагом

self.weights += learning_rate * grad_w

self.bias += learning_rate * grad_b

new_train_pred = self.forward(X_train)

train_loss = mse_loss(y_train, new_train_pred)

test_pred = self.forward(X_test)

test_loss = mse_loss(y_test, test_pred)

train_errors.append(train_loss)

test_errors.append(test_loss)

prev_train_loss = train_loss

```

        if train_loss < 1e-5:
            break

    return train_errors, test_errors, epoch + 1

# =====
# ОЦЕНКА КАЧЕСТВА
# =====
def accuracy_score(y_true, y_pred):
    return np.mean(y_true == y_pred)

def print_dataset_examples(X, y, count=10):
    """
    Печатает несколько примеров из таблицы истинности.
    """
    print("\nПервые примеры таблицы истинности:")
    for i in range(min(count, len(X))):
        print(f"{X[i].astype(int)} -> {int(y[i][0])}")

def evaluate_model(model, X_train, y_train, X_test, y_test, title=""):
    """
    Оценка модели на train и test.
    """
    train_probs = model.predict_proba(X_train)
    train_preds = model.predict(X_train)
    test_probs = model.predict_proba(X_test)
    test_preds = model.predict(X_test)

    train_acc = accuracy_score(y_train, train_preds)
    test_acc = accuracy_score(y_test, test_preds)

    print(f"\n===== {title} =====")
    print(f"Точность на обучающей выборке: {train_acc:.4f}")
    print(f"Точность на тестовой выборке: {test_acc:.4f}")
    print("\nИтоговые веса:")
    for i, w in enumerate(model.weights.flatten(), start=1):
        print(f"w{i} = {w:.6f}")
    print(f"bias = {model.bias[0]:.6f}")

    return train_acc, test_acc, train_probs, test_probs

# =====
# ГРАФИКИ
# =====
def plot_errors(train_errors, test_errors, title):
    plt.figure(figsize=(10, 6))
    plt.plot(train_errors, label="Ошибка на train")

```

```

plt.plot(test_errors, label="Ошибка на test")
plt.xlabel("Эпоха")
plt.ylabel("MSE ошибка")
plt.title(title)
plt.legend()
plt.grid(True)
plt.show()

```

```

# =====
# РЕЖИМ ФУНКЦИОНИРОВАНИЯ
# =====
def interactive_mode(model, n_inputs):
    """
    Ручной ввод вектора из 0 и 1.
    """
    print("\n===== РЕЖИМ ФУНКЦИОНИРОВАНИЯ =====")
    print(f"Введите {n_inputs} чисел 0 или 1 через пробел.")
    print("Чтобы выйти, введите: exit")

    while True:
        user_input = input("\nВаш вектор: ").strip()

        if user_input.lower() == "exit":
            print("Выход из режима функционирования.")
            break

        parts = user_input.split()

        if len(parts) != n_inputs:
            print(f"Ошибка: нужно ввести ровно {n_inputs} чисел.")
            continue

        try:
            vector = np.array([int(x) for x in parts], dtype=float)
        except ValueError:
            print("Ошибка: вводите только числа 0 или 1.")
            continue

        if not np.all(np.isin(vector, [0, 1])):
            print("Ошибка: допустимы только 0 и 1.")
            continue

        vector = vector.reshape(1, -1)
        probability = model.predict_proba(vector)[0][0]
        predicted_class = int(probability >= 0.5)

        print(f"Вероятность класса 1: {probability:.6f}")
        print(f"Округленный класс: {predicted_class}")

# =====

```



```

# ОСНОВНАЯ ПРОГРАММА
# =====
def main():
    print("Лабораторная работа №4")
    print("Классификация линейно разделимых образов однослойным персептроном")
    print(f"Вариант 25: n = {N_INPUTS}, логическая функция OR")

    # 1. Генерация таблицы истинности
    X, y = generate_truth_table(N_INPUTS)
    print(f"\nВсего наборов в таблице истинности: {len(X)}")
    print_dataset_examples(X, y, count=10)

    # 2. Деление на train/test
    X_train, X_test, y_train, y_test = train_test_split_manual(
        X, y, train_ratio=TRAIN_RATIO, seed=RANDOM_SEED
    )

    print(f"\nРазмер обучающей выборки: {len(X_train)}")
    print(f"Размер тестовой выборки: {len(X_test)}")

    # 3. Эксперименты с фиксированным шагом
    fixed_results = []

    for lr in FIXED_LEARNING_RATES:
        np.random.seed(RANDOM_SEED)
        model = SingleLayerPerceptron(N_INPUTS)

        train_errors, test_errors, epochs = model.train_fixed_lr(
            X_train, y_train, X_test, y_test,
            learning_rate=lr,
            max_epochs=MAX_EPOCHS
        )

        train_acc, test_acc, _, _ = evaluate_model(
            model, X_train, y_train, X_test, y_test,
            title=f"Фиксированный шаг обучения alpha = {lr}"
        )

        print(f"Количество эпох: {epochs}")

        plot_errors(
            train_errors,
            test_errors,
            title=f"Фиксированный шаг alpha = {lr}"
        )

        fixed_results.append({
            "lr": lr,
            "epochs": epochs,
            "train_acc": train_acc,
            "test_acc": test_acc,
            "model": model,

```

```

        "train_errors": train_errors,
        "test_errors": test_errors
    })

# 4. Эксперимент с адаптивным шагом
np.random.seed(RANDOM_SEED)
adaptive_model = SingleLayerPerceptron(N_INPUTS)

adaptive_train_errors, adaptive_test_errors, adaptive_epochs = adaptive_model.train_adaptive_lr(
    X_train, y_train, X_test, y_test,
    initial_lr=ADAPTIVE_INITIAL_LR,
    max_epochs=MAX_EPOCHS
)

adaptive_train_acc, adaptive_test_acc, _, _ = evaluate_model(
    adaptive_model, X_train, y_train, X_test, y_test,
    title=f"Адаптивный шаг обучения, начальный alpha = {ADAPTIVE_INITIAL_LR}"
)

print(f"Количество эпох: {adaptive_epochs}")

plot_errors(
    adaptive_train_errors,
    adaptive_test_errors,
    title=f"Адаптивный шаг, начальный alpha = {ADAPTIVE_INITIAL_LR}"
)

# 5. Итоговое сравнение
print("\n===== ИТОГОВОЕ СРАВНЕНИЕ =====")
for result in fixed_results:
    print(
        f"Фиксированный alpha = {result['lr']}: ",
        f"эпох = {result['epochs']}, ",
        f"train_acc = {result['train_acc']:.4f}, ",
        f"test_acc = {result['test_acc']:.4f}"
    )

print(
    f"Адаптивный шаг: эпох = {adaptive_epochs}, ",
    f"train_acc = {adaptive_train_acc:.4f}, ",
    f"test_acc = {adaptive_test_acc:.4f}"
)

# 6. Выбираем лучшую модель по точности на test
best_fixed = max(fixed_results, key=lambda r: r["test_acc"])
best_model = adaptive_model if adaptive_test_acc >= best_fixed["test_acc"] else best_fixed["model"]

if best_model is adaptive_model:
    print("\nЛучшая модель: адаптивный шаг.")
else:
    print("\nЛучшая модель: фиксированный шаг.")

```

```

# 7. Проверка обобщающей способности на нескольких примерах из test
print("\n===== НЕСКОЛЬКО ПРЕДСКАЗАНИЙ НА TEST =====")
test_predictions = best_model.predict(X_test)
test_probabilities = best_model.predict_proba(X_test)

for i in range(min(15, len(X_test))):
    print(
        f"Вход: {X_test[i].astype(int)} | "
        f"Истинный класс: {int(y_test[i][0])} | "
        f"Вероятность: {test_probabilities[i][0]:.6f} | "
        f"Предсказание: {int(test_predictions[i][0])}"
    )

# 8. Режим функционирования
interactive_mode(best_model, N_INPUTS)

if __name__ == "__main__":
    main()

```

Результат:

Лабораторная работа №4

Классификация линейно разделимых образов однослойным персептроном

Вариант 25: $n = 11$, логическая функция OR

Всего наборов в таблице истинности: 2048

Первые примеры таблицы истинности:

```

[0 0 0 0 0 0 0 0 0 0 0] -> 0
[0 0 0 0 0 0 0 0 0 0 1] -> 1
[0 0 0 0 0 0 0 0 0 1 0] -> 1
[0 0 0 0 0 0 0 0 0 1 1] -> 1
[0 0 0 0 0 0 0 0 1 0 0] -> 1
[0 0 0 0 0 0 0 0 1 0 1] -> 1
[0 0 0 0 0 0 0 0 1 1 0] -> 1
[0 0 0 0 0 0 0 0 1 1 1] -> 1
[0 0 0 0 0 0 0 1 0 0 0] -> 1
[0 0 0 0 0 0 0 1 0 0 1] -> 1

```

Размер обучающей выборки: 1433

Размер тестовой выборки: 615

===== Фиксированный шаг обучения $\alpha = 0.1$ =====

Точность на обучающей выборке: 0.9993

Точность на тестовой выборке: 1.0000

Итоговые веса:

$w_1 = 0.423390$

$w_2 = 0.798009$

$w_3 = 0.657201$

$w_4 = 0.567063$

$w_5 = 0.309351$

$w_6 = 0.284650$

$w_7 = 0.226253$

$w_8 = 0.753627$

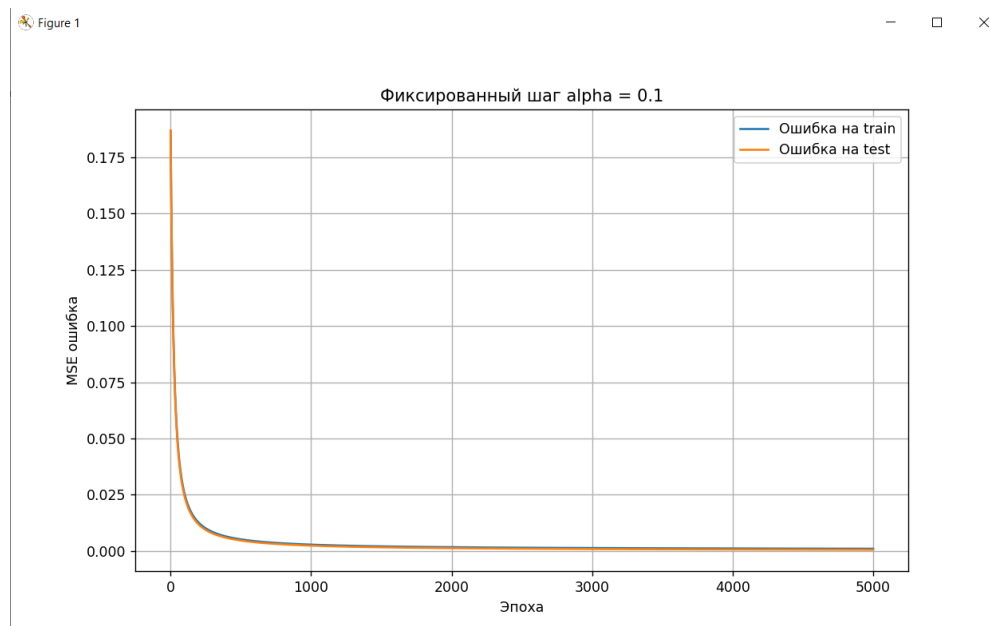
$w_9 = 0.562234$

$w_{10} = 0.645730$

$w_{11} = 0.211761$

$\text{bias} = 1.721070$

Количество эпох: 5000



===== **Фиксированный шаг обучения $\alpha = 0.3$** =====

Точность на обучающей выборке: 0.9993

Точность на тестовой выборке: 1.0000

Итоговые веса:

$w_1 = 0.513400$

$w_2 = 0.849974$

$w_3 = 0.722209$

$w_4 = 0.641398$

$w_5 = 0.420430$

$w_6 = 0.389886$

$w_7 = 0.337254$

$w_8 = 0.814826$

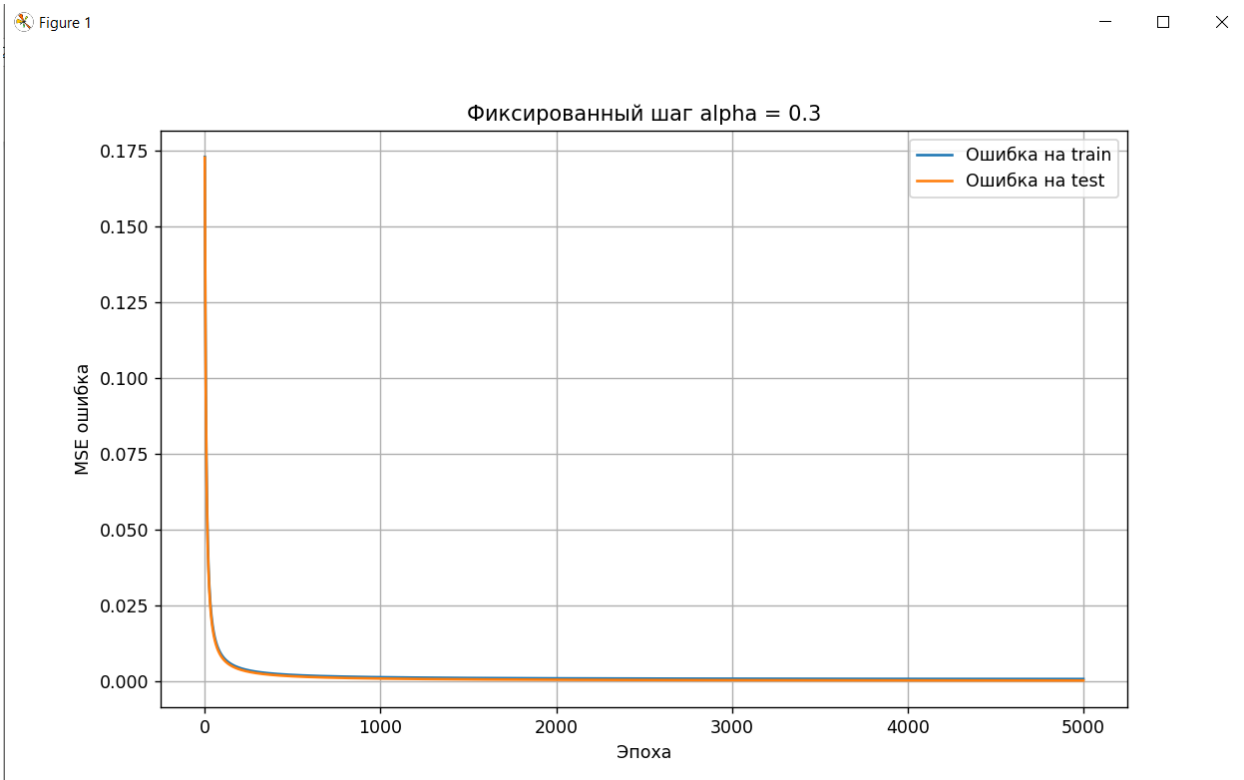
$w_9 = 0.634633$

$w_{10} = 0.715731$

$w_{11} = 0.326712$

$\text{bias} = 1.939101$

Количество эпох: 5000



===== **Фиксированный шаг обучения $\alpha = 0.5$** =====

Точность на обучающей выборке: 0.9993

Точность на тестовой выборке: 1.0000

Итоговые веса:

$w_1 = 0.557590$

$w_2 = 0.875948$

$w_3 = 0.754468$

$w_4 = 0.678130$

$w_5 = 0.474103$

$w_6 = 0.441209$

$w_7 = 0.390440$

$w_8 = 0.845248$

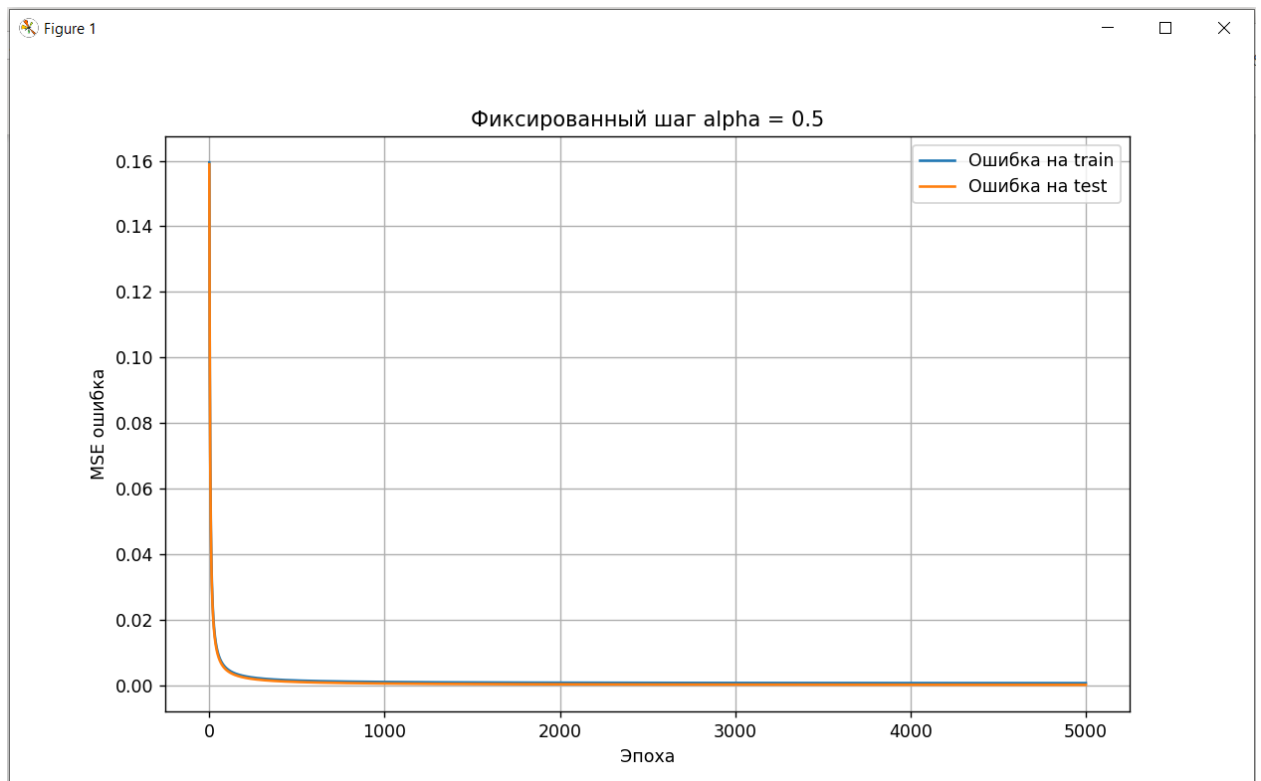
$w_9 = 0.670212$

$w_{10} = 0.750785$

$w_{11} = 0.381421$

$\text{bias} = 2.025463$

Количество эпох: 5000



===== Адаптивный шаг обучения, начальный alpha = 0.5 =====

Точность на обучающей выборке: 1.0000

Точность на тестовой выборке: 1.0000

Итоговые веса:

w1 = 5.787671

w2 = 3.462507

w3 = 5.786774

w4 = 5.788992

w5 = 3.583088

w6 = 5.789030

w7 = 3.257192

w8 = 3.469681

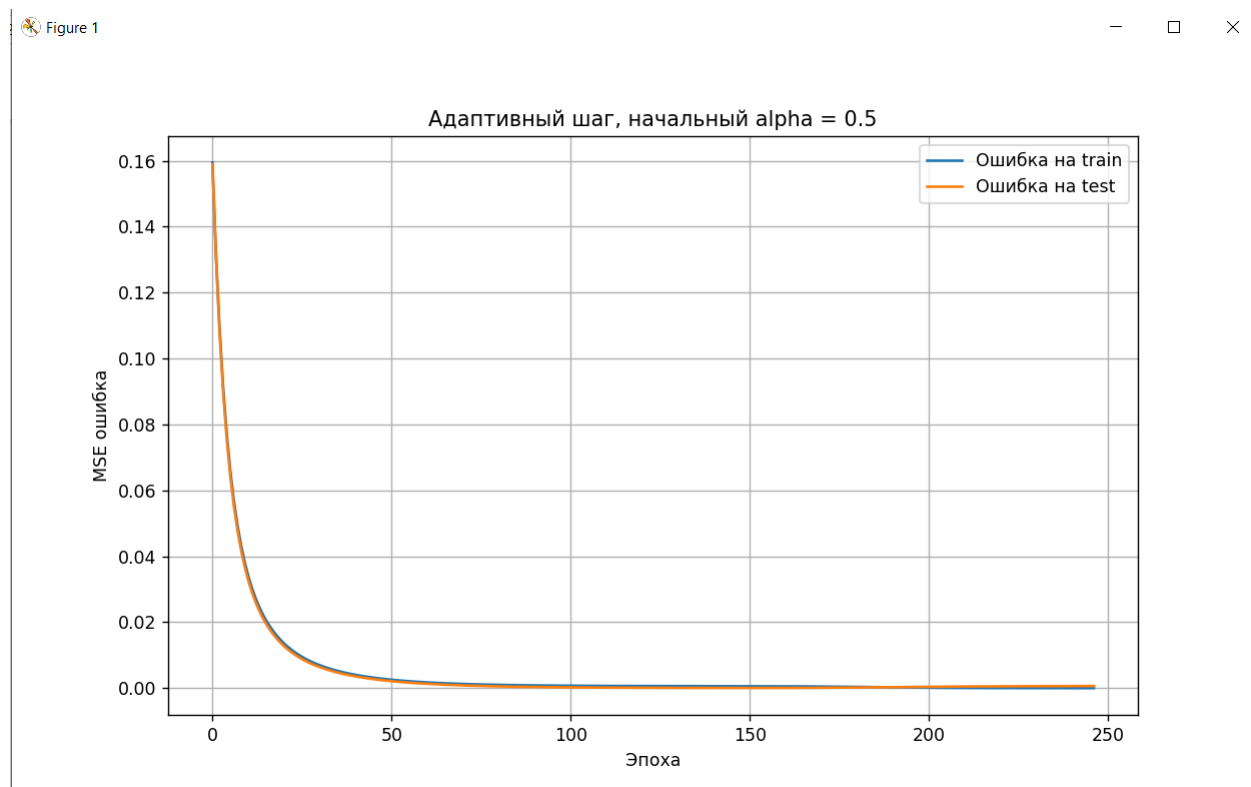
w9 = 3.410954

w10 = 5.790334

w11 = 3.396225

bias = -2.364182

Количество эпох: 247



===== ИТОГОВОЕ СРАВНЕНИЕ =====

Фиксированный alpha = 0.1: эпох = 5000, train_acc = 0.9993, test_acc = 1.0000

Фиксированный alpha = 0.3: эпох = 5000, train_acc = 0.9993, test_acc = 1.0000

Фиксированный alpha = 0.5: эпох = 5000, train_acc = 0.9993, test_acc = 1.0000

Адаптивный шаг: эпох = 247, train_acc = 1.0000, test_acc = 1.0000

Лучшая модель: адаптивный шаг.

===== НЕСКОЛЬКО ПРЕДСКАЗАНИЙ НА TEST =====

Вход: [0 0 0 0 0 0 1 0 1 1] | Истинный класс: 1 | Вероятность: 0.999966 | Предсказание: 1

Вход: [0 0 1 1 0 0 0 1 1 0 0] | Истинный класс: 1 | Вероятность: 1.000000 | Предсказание: 1

Вход: [0 0 1 0 0 0 1 1 1 0 0] | Истинный класс: 1 | Вероятность: 0.999999 | Предсказание: 1

Вход: [1 0 0 0 0 1 0 1 0 1 0] | Истинный класс: 1 | Вероятность: 1.000000 | Предсказание: 1

Вход: [1 0 0 1 0 1 0 0 1 1 1] | Истинный класс: 1 | Вероятность: 1.000000 | Предсказание: 1

Вход: [1 1 0 1 1 1 1 1 0 1 0] | Истинный класс: 1 | Вероятность: 1.000000 | Предсказание: 1

Вход: [0 1 0 1 0 1 1 0 0 1 0] | Истинный класс: 1 | Вероятность: 1.000000 | Предсказание: 1

Вход: [1 1 0 1 0 0 0 0 1 0 0] | Истинный класс: 1 | Вероятность: 1.000000 | Предсказание: 1

Вход: [1 1 1 1 0 0 0 0 0 1 0] | Истинный класс: 1 | Вероятность: 1.000000 | Предсказание: 1

Вход: [0 1 0 1 0 1 1 0 0 0 1] | Истинный класс: 1 | Вероятность: 1.000000 | Предсказание: 1

Вход: [0 1 1 0 1 0 1 0 0 1 1] | Истинный класс: 1 | Вероятность: 1.000000 | Предсказание: 1

Вход: [0 0 0 1 0 0 1 1 1 1 1] | Истинный класс: 1 | Вероятность: 1.000000 | Предсказание: 1

Вход: [0 1 1 0 1 1 1 1 1 1 1] | Истинный класс: 1 | Вероятность: 1.000000 | Предсказание: 1

Вход: [1 1 0 0 1 0 0 1 0 1 1] | Истинный класс: 1 | Вероятность: 1.000000 | Предсказание: 1

Вход: [0 0 1 0 0 0 0 0 0 0 1] | Истинный класс: 1 | Вероятность: 0.998908 | Предсказание: 1

===== РЕЖИМ ФУНКЦИОНИРОВАНИЯ =====

Введите 11 чисел 0 или 1 через пробел.

Чтобы выйти, введите: exit

Ваш вектор: 1 0 0 0 1 1 0 1 1 0 1

Вероятность класса 1: 1.000000

Округленный класс: 1

Ваш вектор: 0 0 0 0 0 0 0 0 0 0

Вероятность класса 1: 0.085945

Округленный класс: 0

Ваш вектор: 0 0 0 0 0 1 1 1 1 1

Вероятность класса 1: 1.000000

Округленный класс: 1

Ваш вектор: 1 1 1 1 1 0 1 1 0 1

Вероятность класса 1: 1.000000

Округленный класс: 1

Соблюдение условий выполнения лабораторной работы

В ходе выполнения лабораторной работы были соблюдены все установленные ограничения и требования.

Для реализации нейронной сети использовались только базовые математические и графические библиотеки языка Python, а именно NumPy и Matplotlib. Библиотека NumPy применялась для выполнения векторных и матричных операций, необходимых при вычислении выходов нейронной сети и обновлении весовых коэффициентов. Библиотека Matplotlib использовалась для построения графиков зависимости ошибки от номера эпохи.

Запрещённые в рамках данной лабораторной работы библиотеки машинного обучения, такие как PyTorch, Keras и другие аналогичные фреймворки, не использовались. Алгоритм обучения однослойного персептрона был реализован вручную, включая вычисление функции активации, ошибки и обновление весов.

Вывод:

В ходе лабораторной работы был реализован однослойный сигмоидальный персептрон для классификации логической функции OR от 11 переменных. Была сгенерирована полная таблица истинности, после чего данные были разделены на обучающую и тестовую выборки. Проведены эксперименты с фиксированным и адаптивным шагом обучения. По графикам ошибки видно, что сеть сходится и успешно обучается. Полученные результаты показывают, что функция OR является линейно разделимой, поэтому однослойный персептрон способен корректно её аппроксимировать и восстанавливать значения функции даже по частичной выборке. Также была подтверждена хорошая обобщающая способность сети на тестовых данных.